UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO

CURSO DE ENGENHARIA INFORMÁTICA

TRABALHO EM GRUPO DE LINGUAGEM DE PROGRAMAÇÃO VI

Tema:

Validação De Dados De Formulário Com Regex E Decoradores, E Criação De Um Sistema De Validação De Dados Para Formulários Web (Simulado), Utilizando Expressões Regulares Para Verificar Formatos E Decoradores Para Aplicar As Validações De Forma Modular.

Grupo Nº: 1

Turma: TL201

Sala: Pavilhão

Orientado por

Moyo Kanivengidi

Uíge, Abril

Integrantes do grupo

1. Bernardo Geraldo Pedro
2. Eduardo Monteiro
3. Miguel Kiala L. Celestino
4. Samuel A. Felix
5. Victor Afonso

Índice

1. Introdução 4
2.Objetivos 4
2.1.Objetivos Gerais 4
2.2.Objetivo específicos 5
3.Regulares (Regex) 5
4. Aplicações Práticas das Regex 5
Tabela1 6
Exemplos1 6
Exemplo2 6
5.Decoradores 6
5.1.Aplicações dos Decoradores na Programação 7
6. Relação entre Regex e Decoradores 7
8.Referências Bibliográficas 13

1. Introdução

No mundo atual da informática, a necessidade de desenvolver sistemas eficientes, seguros e organizados tem levado ao uso de técnicas cada vez mais avançadas dentro das linguagens de programação. A linguagem Python destaca-se nesse contexto por oferecer recursos que facilitam tanto a escrita quanto a manutenção de código, sendo amplamente utilizada em diversas áreas como desenvolvimento web, análise de dados e automação de tarefas.

Entre os recursos mais importantes disponíveis em Python encontram-se as expressões regulares, conhecidas como Regex, e os decoradores, ambos fundamentais para o tratamento de dados e para a organização do código. As expressões regulares permitem trabalhar com padrões de texto de forma precisa, enquanto os decoradores possibilitam modificar o comportamento de funções de maneira simples e elegante.

Desta forma, este trabalho tem como objetivo apresentar de forma clara e detalhada os conceitos de Regex e Decoradores, destacando o seu funcionamento, importância e principais aplicações no contexto da programação.

2.Objetivos

2.1.Objetivos Gerais

Desenvolver um sistema em Python capaz de validar dados d formulários utilizando expressões regulares e decoradores, garantindo organização, reutilização de código e eficiência no trabalho de dados.

2.2.Objetivo específicos

- Compreender o conceito de expressões regulares
- Aplicar Regex na validação de dados
- Validar_email;
- Validar_telefone;
- Validar_data(data);
- Validar_nif(nif);
- Simulação e envio de dados de formulário válidos e inválidos para a função processar cadastro e observe o comportamento.

3.Regulares (Regex)

As expressões regulares, também conhecidas como Regex, representam uma ferramenta extremamente poderosa utilizada na programação para trabalhar com textos, permitindo identificar padrões específicos dentro de uma sequência de caracteres, o que se torna essencial em situações onde é necessário validar ou filtrar informações introduzidas pelo utilizador. Em Python, essa funcionalidade é disponibilizada através do módulo re, que contém diversas funções capazes de verificar, procurar ou substituir padrões em strings de forma eficiente.

O funcionamento das expressões regulares baseia-se na utilização de símbolos especiais que representam diferentes tipos de caracteres, como números, letras ou espaços, permitindo assim a construção de padrões bastante flexíveis e adaptáveis a diferentes situações. Por exemplo, é possível criar um padrão que aceite apenas números com determinada quantidade de dígitos, ou que valide um formato específico de identificação pessoal, o que é bastante útil em sistemas que exigem controlo rigoroso da informação inserida.

Além disso, as Regex permitem não apenas validar dados, mas também realizar operações mais complexas, como extrair partes específicas de um texto ou substituir conteúdos automaticamente, o que contribui significativamente para a automatização de tarefas repetitivas e para a melhoria da eficiência no processamento de dados em aplicações reais.

4. Aplicações Práticas das Regex

As expressões regulares são amplamente utilizadas em diferentes tipos de sistemas, especialmente na validação de dados em formulários, onde é necessário garantir que as informações introduzidas pelos utilizadores seguem um formato correto antes de serem processadas ou armazenadas. Por exemplo, em Angola, pode-se utilizar Regex para validar números de telefone que começam com o dígito 9 e possuem nove números no total, ou ainda para verificar se um número de Bilhete de Identidade segue o padrão exigido.

Para além da validação, as Regex também são utilizadas na pesquisa de palavras dentro de textos extensos, na limpeza de dados ao remover caracteres indesejados, e na transformação de conteúdo, tornando-as uma ferramenta essencial em áreas como análise de dados, desenvolvimento web e processamento de linguagem natural. No entanto, apesar das suas vantagens, é importante referir que a sua utilização exige algum cuidado, pois padrões mal definidos podem gerar erros difíceis de identificar, além de tornar o código menos legível para outros programadores.

Símbolo	Significado
.	Qualquer Caractere
^	Início da string
$	Final da string
*****	Zero ou mais ocorrências
+	Uma ou mais ocorrências
?	Zero ou uma ocorrência
\d	Dígito (0-9)
\W	Letra ou número
\S	Espaço em branco

Tabela1 :Principais símbolos e significados

Exemplos1: Validação de número de telefone (Angola) Nesta imagem temos o import re, que traz para o programa as ferramentas de Regex; re.match, verifica se o texto começa com um padrão.

```
import re
telefone = input( " digite o telefone")
padrao = r"^9\d{8}$"
if re.match(padrao,telefone):
    print("telefone Valido")
else:
    print("telefone inavlido")
```

```
import re
BI = input(" Digite o numero do BI :")

Regra = r"^\d{9}[A-Z]{2}\d{3}$"
if re.match(Regra,BI):
    print(" BI Valido")
else:
    print(" BI Invalido")
```

Exemplo2: Validação do número do BI

5.Decoradores

Os decoradores em Python constituem um recurso avançado que permite modificar ou estender o comportamento de funções sem alterar diretamente o seu código original, sendo particularmente úteis na organização de programas e na reutilização de código. De forma geral, um decorador é uma função que recebe outra função como argumento e devolve uma nova função com funcionalidades adicionais, o que permite adicionar comportamentos de forma dinâmica e elegante.

A importância dos decoradores está relacionada com a sua capacidade de promover uma melhor estruturação do código, evitando a repetição de instruções e facilitando a manutenção de sistemas mais complexos. Em vez de inserir o mesmo código em várias funções, o programador pode criar um decorador que execute essa funcionalidade comum, aplicando-o sempre que necessário, o que contribui para um código mais limpo e organizado.

5.1.Aplicações dos Decoradores na Programação

Na prática, os decoradores são utilizados em diversas situações, como na validação de dados antes da execução de uma função, no controlo de acesso a determinadas funcionalidades, no registo de atividades realizadas pelo sistema e na medição do tempo de execução de processos. Esta versatilidade faz com que os decoradores sejam amplamente utilizados em aplicações reais, especialmente em sistemas web e APIs.

Outra vantagem importante dos decoradores é a sua sintaxe simples e intuitiva, que utiliza o símbolo "@" para indicar que uma função está a ser modificada, tornando o código mais fácil de compreender e organizar.

6. Relação entre Regex e Decoradores

Embora as expressões regulares e os decoradores sejam conceitos distintos, é possível utilizá-los de forma complementar no desenvolvimento de aplicações, especialmente em situações que envolvem validação de dados. Por exemplo, pode-se criar um decorador que utilize Regex para verificar automaticamente se os dados introduzidos pelo utilizador estão corretos antes de permitir a execução de uma determinada função.

Essa combinação permite desenvolver sistemas mais seguros e organizados, uma vez que garante que apenas dados válidos são processados, ao mesmo tempo em que mantém o código estruturado e de fácil manutenção. Dessa forma, a integração dessas duas ferramentas demonstra a flexibilidade da linguagem Python e a sua capacidade de oferecer soluções eficientes para problemas complexos.

Sistema de validação de dados para formulários web (simulado), utilizando expressões regulares para verificar formatos e decoradores para aplicar as validações de forma modular.

```
import re

# ========================

# FUNÇÕES DE VALIDAÇÃO

# ========================

def validar_email(email):

    padrao = r'^[\w\.-]+@[\w\.-]+\.\w+$'

    return re.match(padrao, email)


def validar_telefone(telefone):

    padrao = r'^(\+244)?\s?\(?\d{3}\)?[\s-]?\d{3}[\s-]?\d{3}$'

    return re.match(padrao, telefone)


def validar_data(data):

    padrao = r'^\d{2}-\d{2}-\d{4}$'

    return re.match(padrao, data)


def validar_nif(nif):

    padrao = r'^\d{10}$'

    return re.match(padrao, nif)


# ========================

# DECORADOR

# ========================


def validar_formulario(validacoes):

    def decorador(func):

        def wrapper(**kwargs):

            erros = {}
```

```
        for campo, validador in validacoes.items():
            valor = kwargs.get(campo)

            if not validador(valor):
                erros[campo] = f"{campo} inválido"

        if erros:
            return {"status": "erro", "erros": erros}

        return func(**kwargs)
    return wrapper
return decorador

# ========================
# FUNÇÃO PRINCIPAL
# ========================

@validar_formulario({
    "email": validar_email,
    "telefone": validar_telefone,
    "data": validar_data,
    "nif": validar_nif
})
def processar_cadastro(**dados):
    return {
        "status": "sucesso",
        "mensagem": "Cadastrado com sucesso!",
        "dados": dados
    }
```

```
# ========================
# TESTE
# ========================
print("=== CADASTRO ===")

email = input("Email: ")
telefone = input("Telefone: ")
data = input("Data (DD-MM-AAAA): ")
nif = input("NIF: ")

resultado = processar_cadastro(
    email=email,
    telefone=telefone,
    data=data,
    nif=nif
)
print("\nResultado:")
print(resultado)
```

7.Conclusão

Em suma, as expressões regulares e os decoradores são recursos fundamentais na programação em Python, contribuindo significativamente para a validação de dados, organização do código e desenvolvimento de aplicações mais eficientes. As Regex destacam-se pela sua capacidade de trabalhar com padrões de texto de forma precisa, enquanto os decoradores permitem uma abordagem mais modular e reutilizável no desenvolvimento de software.

Apesar de apresentarem alguma complexidade inicial, o domínio dessas ferramentas proporciona inúmeras vantagens, tornando o programador mais preparado para enfrentar desafios reais no desenvolvimento de sistemas. Assim, o estudo contínuo e a prática desses conceitos são essenciais para alcançar um nível mais avançado na programação em Python.

8.Referê ncias Bibliográficas

LUTZ, Mark. Aprendendo Python. O'Reilly Media, 2013.

SWEIGART, Al. Automatize Tarefas Maçantes com Python. Novatec, 2016.

FRIEDL, Jeffrey. Mastering Regular Expressions. O'Reilly, 2006.

RAMALHO, Luciano. Python Fluente. Novatec, 2023.

Python Software Foundation. Documentação oficial do Python. Disponível em: https://docs.python.org